

Techniques for 3D Mapping

Autonomous Navigation of Mobile Robots

Anna Semakova

a.semakova@gmail.com

15.05.2025

Why 3D Representations

- Robots live in the 3D world
- 2D maps have been applied successfully for navigation tasks such as localization
- Reliable collision avoidance and path planning, however, requires accurate 3D models
- How to represent the 3D structure of the environment?

Popular Representations

- Point clouds
- Voxel grids
- Surface maps
- Meshes
- ...

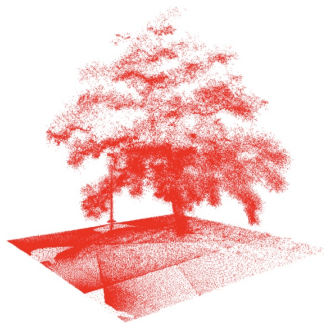
Point Clouds

Pro

- No discretization of data
- Mapped area not limited

Contra

- Unbounded memory usage
- No direct representation of free or unknown space



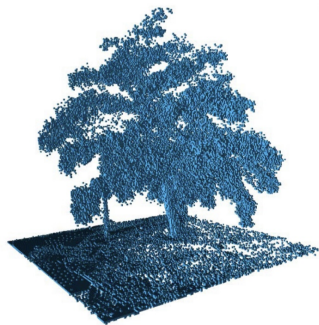
3D Voxel Grids

Pro

- Volumetric representation
- Constant access time
- Probabilistic update

Contra

- Memory requirement – complete map is allocated in memory
- Extent of the map has to be known/guessed
- Discretization errors



2.5D Maps

“Height Maps”

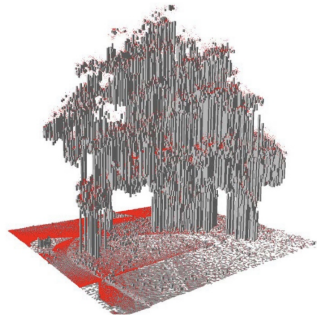
Average over all scan points that fall into a cell

Pro

- Memory efficient
- Constant time access

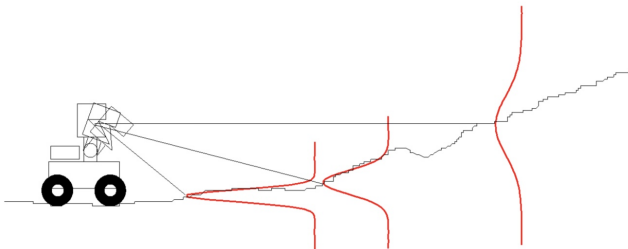
Contra

- Non-probabilistic
- No distinction between free and unknown space



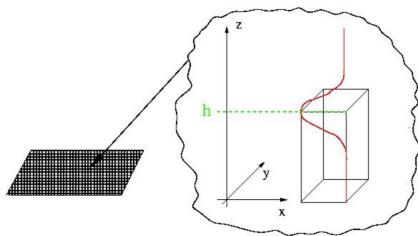
Elevation Maps

- 2D grid that stores an estimated height (elevation) for each cell
- Typically, the uncertainty increases with measured distance



Elevation Maps

- 2D grid that stores an estimated height (elevation) for each cell
- Typically, the uncertainty increases with measured distance
- Kalman update to estimate the elevation



Elevation Maps

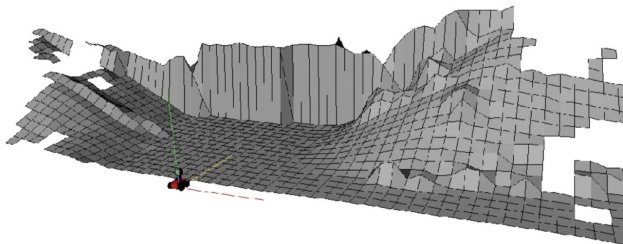
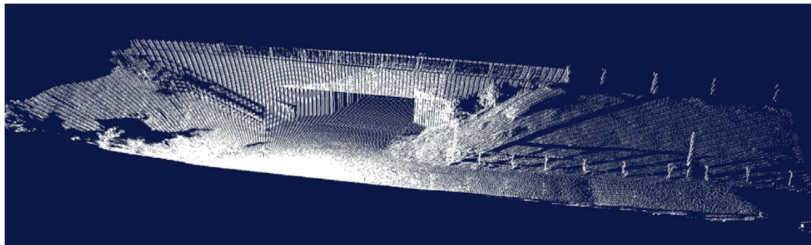
Pro

- 2.5D representation (vs. full 3D grid)
- Constant time access
- Probabilistic estimate about the height

Contra

- No vertical objects
- Only one level is represented

Typical Elevation Map

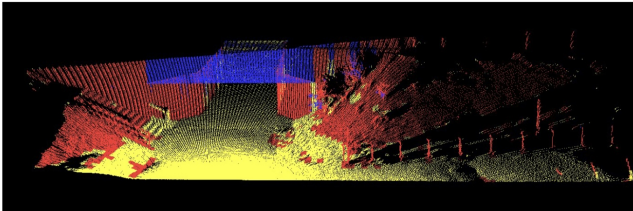


Extended Elevation Maps

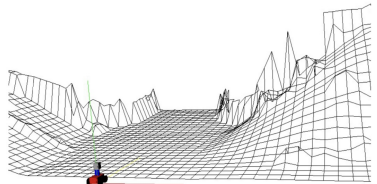
- Identify
 - Cells that correspond to vertical structures
 - Cells that contain gaps
- Check whether the variance of the height of all data points is large for a cell
- If so, check whether the corresponding point set contains a gap exceeding the height of the robot (“gap cell”)

Example

Extended Elevation Map

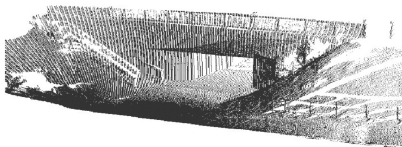


- Cells with vertical objects (red)
- Data points above a big vertical gap (blue)
- Cells seen from above (yellow)
- Use gap cells to determine traversability

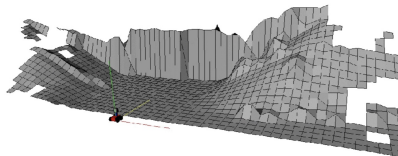


Extended elevation map

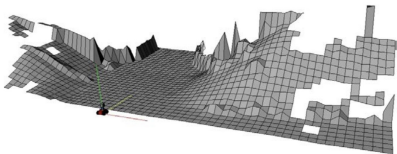
Types of Terrain Maps



Point cloud

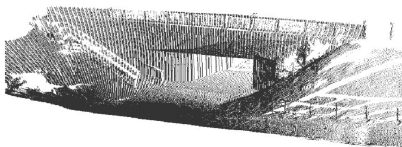


Standard elevation map

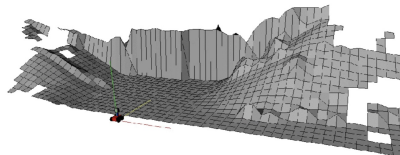


Extended elevation map

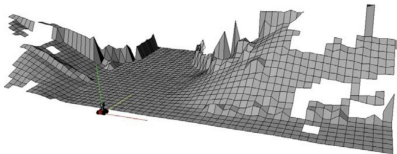
Types of Terrain Maps



Point cloud



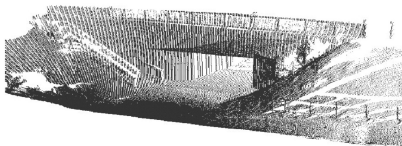
Standard elevation map



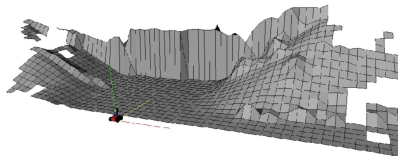
Extended elevation map

- + Planning with underpasses possible (cells with vert. gaps)
- No paths passing under and crossing over bridges possible (only one level per grid cell)

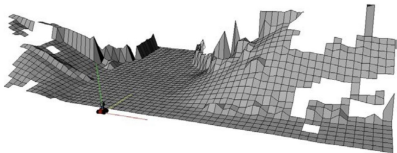
Types of Terrain Maps



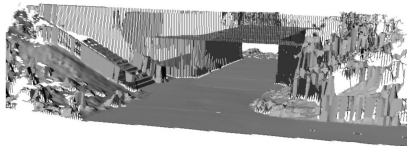
Point cloud



Standard elevation map

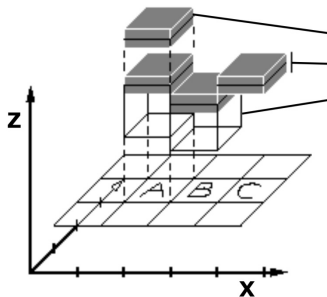


Extended elevation map



Multi-level surface map

MLS Map Representation



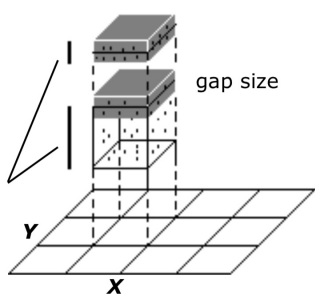
Each 2D cell stores various patches consisting of

- The height mean μ
- The height variance σ
- The depth value d

Note

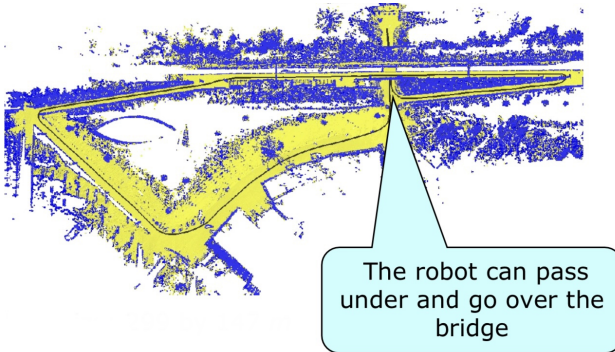
- A patch can have no depth (flat objects, e.g., floor)
- A cell can have one or many patches (vertical gap cells, e.g., bridges)

From Point Clouds to MLS Maps



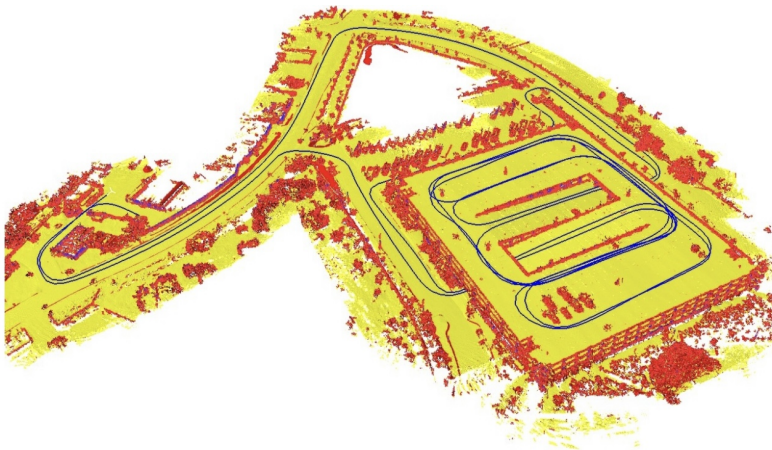
- Determine the cell for each 3D point
 - Compute vertical intervals
 - Classify into vertical (> 10 cm) and horizontal intervals
-
- Apply Kalman update to estimate the height based on all data points for the horizontal intervals
 - Take the mean and variance of the highest measurement for the vertical intervals

Results



- Map size— 299 by 147 m
- Cell resolution – 10 cm
- Number of data points – 45,000,000

MLS Map of the Parking Garage



MLS Maps

Pro

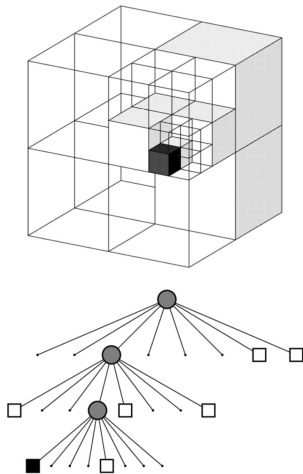
- Can represent multiple surfaces per cell

Contra

- No representation of unknown areas
- No volumetric representation but a discretization in the vertical dimension
- Localization in MLS maps is not straightforward

Octree-based Representation

- Tree-based data structure
- Recursive subdivision of the space into octants
- Volumes allocated as needed
- “Smart 3D grid”



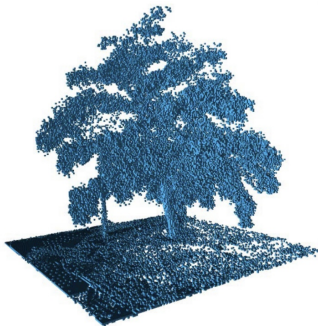
Octrees

Pro

- Full 3D model
- Probabilistic
- Inherently multi-resolution
- Memory efficient

Contra

- Implementation can be tricky (memory, update, map files, ...)



OctoMap Framework

- Based on octrees
- Probabilistic, volumetric representation of occupancy including unknown
- Supports multi-resolution map queries
- Memory efficient
- Compact map files
- Open source implementation as C++ library available at <https://octomap.github.io/>

Probabilistic Map Update

- Occupancy modeled as recursive **binary Bayes filter**

$$Bel(m_t^{[xyz]}) = \left[1 + \frac{1 - P(m_t^{[xyz]} | z_t, u_{t-1})}{P(m_t^{[xyz]} | z_t, u_{t-1})} \cdot \frac{P(m_t^{[xyz]})}{1 - P(m_t^{[xyz]})} \cdot \frac{1 - Bel(m_{t-1}^{[xyz]})}{Bel(m_{t-1}^{[xyz]})} \right]^{-1}$$

- Efficient update using **log-odds** notation

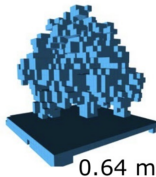
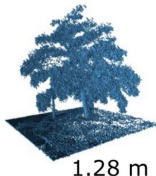
Probabilistic Map Update

- Clamping policy ensures updatability

$$Bel(m_t^{[xyz]}) \in [l_{\min}, l_{\max}]$$

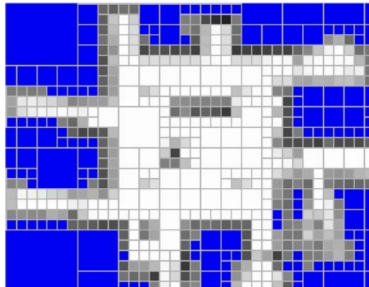
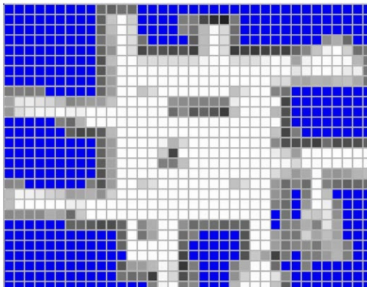
- Multi-resolution queries using

$$Bel(n) = \max_{i=1\dots 8} Bel(n_i), \quad n_i \in \text{children}(n)$$



Lossless Map Compression

- Lossless pruning of nodes with identical children
- Can lead to high compression ratios

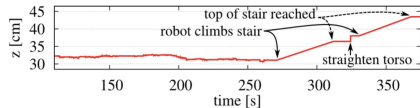
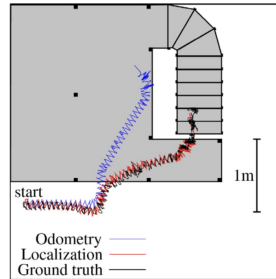


6D Localization with a Humanoid

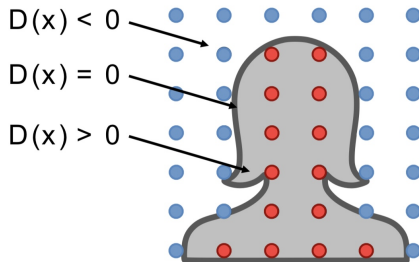
https://wiki.ros.org/humanoid_localization



Goal – Accurate pose tracking while walking and climbing stairs

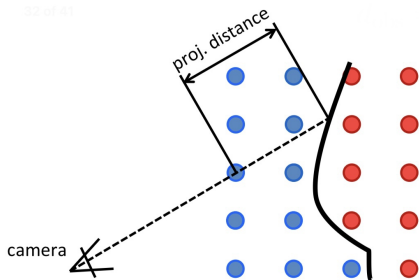


Signed Distance Function (SDF)



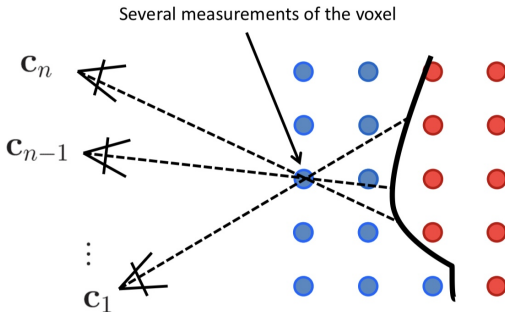
Signed Distance Function (SDF)

- Compute SDF from a depth image
- Measure distance of each voxel to the observed surface
- Can be done in parallel for all voxels (GPU)
- Becomes very efficient by only considering a small interval around the endpoint (truncation)



Signed Distance Function (SDF)

- Calculate weighted average over all measurements for every voxel
- Assume known camera poses



$$D \leftarrow \frac{WD + wd}{W + w}$$

$$W \leftarrow W + w$$

Visualizing Signed Distance Fields

Common approaches to iso surface extraction

1. Ray casting (GPU, fast)

For each camera pixel, shoot a ray and search for zero crossing

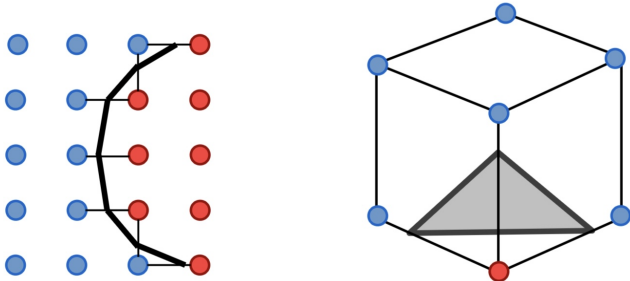
2. Polygonization (CPU, slow)

E.g., using the marching cubes algorithm

Advantage – outputs triangle mesh

Mesh Extraction using Marching Cubes

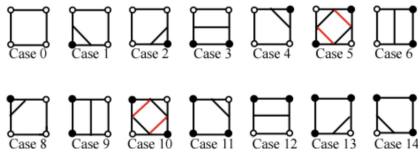
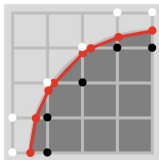
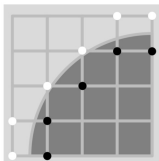
- Find zero-crossings in the signed distance function by interpolation



Marching Cubes

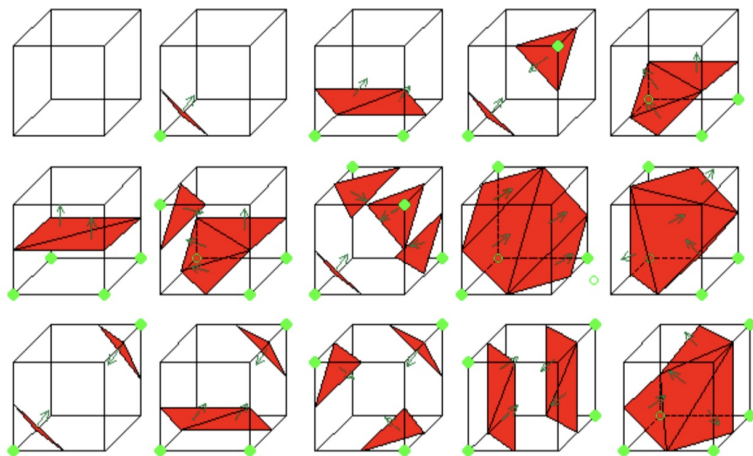
If we are in 2D — **Marching squares**

- Evaluate each cell separately
- Check which edges are inside / outside
- Generate triangles according to 16 lookup tables
- Locate vertices using least squares



Marching Cubes

3D



Signed Distance Functions

Pro

- Full 3D model
- Sub-pixel accuracy
- Fast (graphics card) implementation

Contra

- Space consuming voxel grid

Summary

- Different 3D map representations exist
- The best model always depends upon the corresponding application
- We discussed surface models and voxel representations
- Surface models support a traversability analysis
- Voxel representations allow for a full 3D representation
- Octrees are a probabilistic representation; they are inherently multi-resolution
- Signed distance functions also use three-dimensional grids but allow for a sub-pixel accuracy representation of the surface
- Note – there also is a PointCloud Library for directly dealing with point clouds